

# Final Report: 5-Stage Pipelined RISC-V Processor

## Team Members:

- Om Dave (CO22BTECH11006)
- Ashutosh Bhatta (MA22BTECH11007)

## Summary

This report documents the implementation of a **5-stage pipelined RISC-V processor** as an extension to an existing single-cycle RISC-V simulator. The project implements hazard detection, data forwarding, branch prediction, and complete undo/redo functionality for cycle-level debugging. We use the **Ripes simulator** as a reference for validation and comparison.

**Final Status:** Core pipeline implementation complete with advanced features (forwarding, hazard detection, branch prediction, and undo/redo) along with a functional data cache.

## 1. Project Scope and Design Plan

### 1.1 Objectives

Transform the existing single-cycle execution model into a **classic 5-stage RISC-V pipeline** (IF → ID → EX → MEM → WB) with:

1. **Pipelined Architecture** - Full 5-stage pipeline with inter-stage registers
2. **Data Hazard Management** - Detection, forwarding, and stalling mechanisms
3. **Control Hazard Management** - Branch prediction and pipeline flushing
4. **Comprehensive Statistics** - CPI, stall cycles, mispredictions, bubbles
5. **Debugging Support** - Cycle-level undo/redo with complete state restoration
6. **Configuration Flexibility** - Runtime selection of pipeline features

## 2. Implementation Details

### 2.1 Core Pipeline Implementation **COMPLETED**

#### Pipeline Stage Execution

Each clock cycle executes all 5 stages in **reverse order** to prevent data races:

#### Instruction Fetch (IF)

- Fetches instruction from `program_counter_`
- Writes to `IF/ID` register if not stalled
- Updates PC by +4 (unless branch redirection occurs)

#### Instruction Decode (ID)

- Decodes instruction fields (opcode, rs1, rs2, rd, funct3, funct7)
- Generates control signals via `RVSSControlUnit`
- Reads source registers from register file
- Generates immediate values
- **Hazard detection:** Checks for load-use hazards
- **Branch prediction:** Consults predictor for branch instructions

### Execute (EX)

- Performs ALU operations
- **Data forwarding:** Applies forwarding from EX/MEM and MEM/WB
- Resolves branch conditions
- Handles special instructions (float, CSR, syscalls)

### Memory Access (MEM)

- Reads/writes memory for load/store instructions
- Updates branch predictor with actual branch outcome
- Flushes pipeline on branch misprediction

### Write Back (WB)

- Writes results to register file (GPR/FPR/CSR)
- Records all register changes for undo/redo
- Increments `instructions_retired_`

---

## 2.2 Hazard Detection and Stalling **COMPLETED**

Detects **Read-After-Write (RAW)** hazards, specifically **load-use** hazards:

### Stalling mechanism:

- Sets `pc_write_ = false` and `if_id_write_ = false`
  - Inserts bubble (NOP) into `ID/EX` register
  - Increments `bubbles_inserted_` counter
- 

## 2.3 Data Forwarding **COMPLETED**

## 2.4 Branch Prediction **COMPLETED**

Supports **three prediction modes**:

1. **No Prediction (mode = 0):** Always predict not-taken
2. **Static Prediction (mode = 1):** Configurable policy (predict-taken or predict-not-taken)
3. **Dynamic Prediction (mode = 2):** 2-bit bimodal predictor with configurable counter bits

### Prediction Flow:

- **ID Stage:** Get prediction, redirect fetch if predicted taken
- **MEM Stage:** Compare actual outcome with prediction, update predictor
- **Misprediction:** Flush IF and ID stages, increment `branch_mispredictions_`

---

## 2.5 Undo/Redo Functionality **COMPLETED**

Implements **cycle-level undo/redo** with complete state restoration:

---

## 2.6 Configuration System **COMPLETED**

### VM Types:

```
enum class VmTypes {
    SINGLE_STAGE,           // Original single-cycle VM
    MULTI_STAGE,            // Basic 5-stage (no
forwarding/hazard)
    MULTI_STAGE_WITH_FORWARDING, // With data forwarding
    MULTI_STAGE_WITH_HAZARD_DETECTION, // With hazard detection
    MULTI_STAGE_WITH_BOTH      // With both forwarding and
hazard detection
};
```

### Global Configuration Flags:

```
namespace globals {
    extern bool pipeline_forwarding_enabled;
    extern bool pipeline_hazard_detection_enabled;
    extern int branch_prediction_mode;
    extern int static_branch_policy;
    extern int branch_prediction_bits;
}
```

PROF

---

## 3. Statistics and Metrics

The pipeline tracks comprehensive execution statistics:

| Metric                             | Description                          | Implementation                             |
|------------------------------------|--------------------------------------|--------------------------------------------|
| <code>total_cycles_</code>         | Total clock cycles executed          | Incremented in <code>Step()</code>         |
| <code>instructions_retired_</code> | Instructions that completed WB stage | Incremented in <code>WB()</code>           |
| <code>bubbles_inserted_</code>     | Pipeline stalls due to hazards       | Incremented in <code>InsertBubble()</code> |

| Metric                 | Description                  | Implementation                        |
|------------------------|------------------------------|---------------------------------------|
| branch_mispredictions_ | Incorrect branch predictions | Incremented in MEM( )                 |
| CPI                    | Cycles Per Instruction       | total_cycles_ / instructions_retired_ |

Sample Output:

```
=====
Pipeline Execution Statistics:
Total cycles: 1250
Instructions retired: 1000
Branch mispredictions: 45
Bubbles inserted: 32
CPI (Cycles Per Instruction): 1.25
=====
```

4. Testing and Verification

4.1 Test Programs

Pipeline Test (pipeline\_test1.s):

```
addi x1, x0, 1      # x1 = 1
addi x2, x0, 2      # x2 = 2
addi x3, x0, 3      # x3 = 3
addi x4, x0, 4      # x4 = 4

add x5, x1, x2      # x5 = 3
add x6, x3, x4      # x6 = 7
sub x7, x6, x5      # x7 = 7 - 3 = 4
and x8, x5, x6      # x8 = 3 & 7 = 3

# End
nop
nop
```

4.2 Validation Against Ripes

All test programs verified against Ripes simulator outputs:

- Register state after execution
- Memory state
- Cycle count and CPI

- Number of stalls and mispredictions

### Example Comparison:

w/o forwarding or hazard detection

```
=====
Pipeline Execution Statistics:
Total cycles: 14
Instructions retired: 10
Branch mispredictions: 0
Bubbles inserted: 0
CPI (Cycles Per Instruction): 1.4
=====
```

### Execution info

|                  |                                  |
|------------------|----------------------------------|
| Cycles:          | <input type="text" value="14"/>  |
| Instrs. retired: | <input type="text" value="10"/>  |
| CPI:             | <input type="text" value="1.4"/> |

```
"x1" : "0x0000000000000001",
"x2" : "0x0000000000000002",
"x3" : "0x0000000000000003",
"x4" : "0x0000000000000004",
"x5" : "0x0000000000000003",
"x6" : "0x0000000000000003",
```

PROF

|    |    |                    |
|----|----|--------------------|
| x1 | ra | 0x0000000000000001 |
| x2 | sp | 0x0000000000000002 |
| x3 | gp | 0x0000000000000003 |
| x4 | tp | 0x0000000000000004 |
| x5 | t0 | 0x0000000000000003 |
| x6 | t1 | 0x0000000000000003 |

## w/o hazard detection

|    |    |                    |
|----|----|--------------------|
| x2 | sp | 0x0000000000000002 |
| x3 | gp | 0x0000000000000003 |
| x4 | tp | 0x0000000000000004 |
| x5 | t0 | 0x0000000000000003 |
| x6 | t1 | 0x0000000000000007 |
| x7 | t2 | 0x0000000000000004 |
| x8 | s0 | 0x0000000000000003 |

### 4.3 Automated Pipeline Verification

To ensure the multi-stage pipeline behaves identically to the single-stage reference (functionally), we implemented an automated verification script: `test/verify_pipeline.py`.

#### How it works:

1. Runs the simulator in **Single-Stage** mode on a given assembly file.
2. Captures the final register state (General Purpose and Floating Point registers) into a JSON dump.
3. Runs the simulator in **Multi-Stage** mode (with full features enabled) on the same file.
4. Captures the final register state.
5. Compares the two register dumps entry-by-entry.

#### Usage:

```
python3 test/verify_pipeline.py examples/pipeline_test1.s
```

This ensures that despite the complexity of pipelining, forwarding, and hazard detection, the architectural state remains correct.

### 4.4 Branch Prediction Analysis

We developed a comprehensive testing suite `test/branch_pattern_test.py` to verify and analyze the branch predictor's performance.

#### Testing Methodology:

The script operates in two phases:

1. **Verification Phase:** Runs various branch patterns on the multi-stage pipeline and compares the final register state against the single-stage reference to ensure correctness.

2. **Analysis Phase:** Runs the patterns with different predictor configurations (Static Taken/Not-Taken, Dynamic 1-bit/2-bit) and collects statistics (Accuracy, Mispredictions, CPI).

### Branch Pattern Tests:

The suite uses specific assembly programs (located in `examples/`) designed to stress different prediction scenarios:

- **Always Taken (`branch_pattern_always_taken.s`):** A loop where the branch is always taken. Ideal for "Static Taken" or trained dynamic predictors.
- **Always Not Taken (`branch_pattern_never_taken.s`):** A loop where the branch is never taken. Ideal for "Static Not-Taken".
- **Alternating (`branch_pattern_tntn.s`):** Taken, Not-Taken, Taken, Not-Taken... Hard for static predictors, requires 2-bit dynamic history.
- **Pattern TTNN (`branch_pattern_ttnn.s`):** Taken, Taken, Not-Taken, Not-Taken... Tests the predictor's ability to learn short repeating patterns.
- **Biased (`branch_pattern_biased_taken.s`):** Random-like pattern but with a 75% bias towards Taken. Tests the predictor's adaptability.

## 5. Challenges Faced

### 5.1 Synchronization Between Pipeline Stages

### 5.2 Undo/Redo Complexity

Pipeline has multiple instructions in-flight; reversing a single cycle affects multiple instructions' states.

So we Implemented comprehensive `StepDelta` structure that captures:

- Complete snapshots of all 4 pipeline registers (before and after)
- All architectural register changes
- All memory changes
- Branch predictor state snapshot

### 5.3 Branch Misprediction Handling

When a branch is mispredicted, wrong-path instructions may have already modified pipeline state.

#### Solution:

- Prediction made in ID stage
- Resolution in MEM stage
- Flush IF and ID stages on misprediction (no architectural changes yet)
- Track `predicted_taken` flag through pipeline

---

## 6. Additional Feature - Cache Implementation

We added a configurable, basic set-associative cache to the simulator. The cache implementation is in `include/vm/cache/cache.h` and `src/vm/cache/cache.cpp`. It is integrated with the

`MemoryController` (see `include/vm/memory_controller.h`) and can be enabled at runtime via the configuration flags in `config.h`.

## High-level design

- Type: set-associative cache with configurable associativity and line size.
- Stored per-line metadata: `CacheLine` (state, tag, data vector of bytes).
- Per-set container: `CacheSet` holds `associativity` and a vector of `CacheLine`.
- Policies: replacement (LRU/FIFO/Random enum available, although only LRU has been implemented), write-hit (WriteThrough / WriteBack), write-miss (NoWriteAllocate / WriteAllocate).

## Configuration

- `CacheConfig` (in `cache.h`) contains: `size` (bytes), `lines`, `associativity`, `words_per_line`, `replacement_policy`, `cache_type` (Instruction/Data), `write_hit_policy`, `write_miss_policy`.
- The `MemoryController` constructs a `cache::Cache` instance from `vm_config::config` when the first memory access happens and `cache_enabled` is true. Config fields used are `cache_capacity`, `cache_block_size` and `cache_associativity`.

## Address mapping and bookkeeping

- Line size (`line_size_`) is derived from the config (either `size / lines` or `words_per_line * 4`).
- `addressToTag(address)` returns `address / line_size_` (integer tag).
- `addressToSetIndex(address)` returns `(address / line_size_) % set_count_` where `set_count_ = total_lines / associativity`.

## Replacement & eviction

- Replacement is implemented by keeping the most-recently-used line at the front of the vector and rotating on accesses; the victim is chosen as the last element when no Invalid line exists. Dirty victims are written back (byte-by-byte) before being overwritten.

PROF

## Statistics

- `CacheStats` collects `accesses`, `hits`, `misses`. There is a `PrintStats()` helper to log these numbers.

## Integration with the VM

- The `MemoryController` routes `ReadByte` and `WriteByte` through the cache when enabled. It constructs a `cache::Cache` using values from `vm_config::config` (capacity, block size, associativity) so cache creation is deferred until memory is used.

## Dump/Debug output

- `DumpCache` (in `src/utils.cpp`) was updated to produce JSON-formatted dumps under `vm_state/cache_dump.json`. The structure is:



```
{
  "data_cache": {
    "<set_index>": {
      "<line_index>": { "valid": 0|1, "dirty": 0|1, "words":
["0xdeadbeef", "0x01234567", ...] }
      ...
    },
    ...
  },
  "instruction_cache": { /* empty unless dumping an instruction cache
*/ }
}
```

This output converts line bytes into 32-bit hex words readability and includes both valid and dirty bits.

Files of interest (implementation):

- `include/vm/cache/cache.h` — types and public API (Cache, CacheSet, CacheLine, CacheConfig, enums)
- `src/vm/cache/cache.cpp` — implementation of ReadByte, WriteByte, fetchLineToSet, address mapping and eviction
- `include/vm/memory_controller.h` — cache construction and routing of Read/Write operations
- `src/utils.cpp` — `DumpCache()` JSON output

Testing

- The cache was tested by existing homeworks and class tests and pipeline tests; statistics (`PrintStats`) and `vm_state/cache_dump.json` were used to validate hits/misses and that write-back correctly flushed dirty lines to memory.

Known limitations & future improvements

- Replacement policy enum supports LRU/FIFO/Random but current code only uses a LRU scheme.
- Error detection for misconfigured cache parameters (e.g., associativity larger than number of lines, block size being divisible by the word size(4)) is not implemented.
- The existing memory read operation reads and writes one byte at a time, which is inefficient for cache line fills and write-backs, and also reflects in the stats. Future improvements could include implementing block read/write operations to optimize performance.
- The stats will be different from other simulators that implement block read/writes for cache operations and also because we have integrated cache write operations with the existing byte-wise memory interface, which does not distinguish between data cache and instruction cache operations. Testing has been done by observing the cache dump after running various test programs.

```

"data_cache": {
  "0": {
    "00000000000000000000000000000000": {"valid":1, "dirty":0, "words": ["0x10000197", "0x00018193", "0x0001b203", "0x0081b203", "0x01018193", "0x00000513", "0x00018633", "0x02a20263"]}
  },
  "1": {
    "00000000000000000000000000000001": {"valid":1, "dirty":0, "words": ["0x00000593", "0x00b28a63", "0x00063a03", "0x00860613", "0x00158593", "0xfe0000e3", "0x00150513", "0xfe0000e3"]}
  },
  "2": {
    "00000000000000000000000000000010": {"valid":1, "dirty":0, "words": ["0x00000013", "0x00000000", "0x00000000", "0x00000000", "0x00000000", "0x00000000", "0x00000000", "0x00000000"]}
  },
  "3": {
    "0000000010000000000000000000001011": {"valid":1, "dirty":0, "words": ["0x00000000", "0x00000000", "0x00000000", "0x00000000", "0x00000000", "0x00000000", "0x00000000", "0x00000000"]}
  },
  "4": {
    "0000000010000000000000000000001100": {"valid":1, "dirty":0, "words": ["0x00000000", "0x00000000", "0x00000000", "0x00000000", "0x00000000", "0x00000000", "0x00000000", "0x00000000"]}
  },
  "5": {
    "0000000010000000000000000000001101": {"valid":1, "dirty":0, "words": ["0x00000000", "0x00000000", "0x00000000", "0x00000000", "0x00000000", "0x00000000", "0x00000000", "0x00000000"]}
  },
  "6": {
    "0000000010000000000000000000001110": {"valid":1, "dirty":0, "words": ["0x00000000", "0x00000000", "0x00000000", "0x00000000", "0x00000000", "0x00000000", "0x00000000", "0x00000000"]}
  },
  "7": {
    "0000000010000000000000000000001111": {"valid":1, "dirty":0, "words": ["0x00000000", "0x00000000", "0x00000000", "0x00000000", "0x00000000", "0x00000000", "0x00000000", "0x00000000"]}
  }
},

```